GitSummarizer: Project Documentation

1. Introduction

GitSummarizer is an AI-powered web application designed to simplify how developers, students, and product managers interpret codebases. Instead of digging through extensive README files or parsing cryptic git commit logs, GitSummarizer integrates the **GitHub REST API** with large language models to instantly generate beginner-friendly project overviews, key feature highlights, prerequisites, and clear release notes from any public GitHub repository or commit hash.

2. Use Cases

- **Junior Developers & Students:** Quickly understanding open-source repositories and complex codebases without getting overwhelmed by dense technical documentation.
- **Product Managers & Stakeholders:** Reviewing code changes and translation of technical commits into clear, business-friendly release notes.
- **Open Source Maintainers:** Generating quick overviews or changelogs to improve repository onboarding for new contributors.

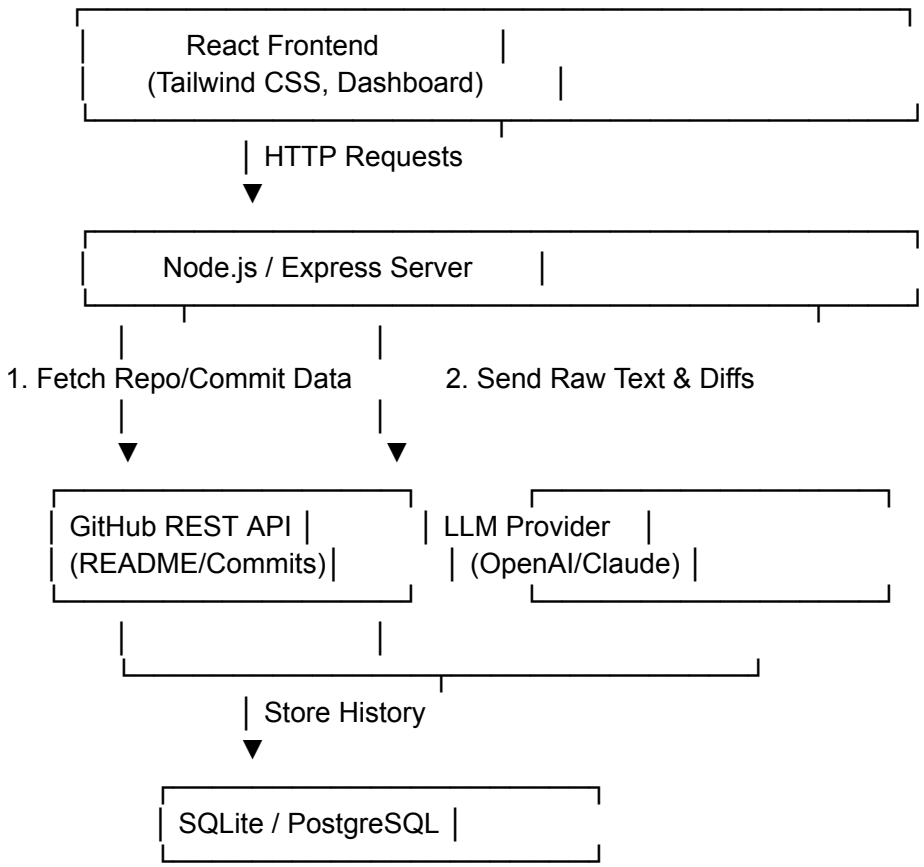
3. Tech Stack

- **Frontend:** React (Vite), Tailwind CSS.
- **Backend:** Node.js, Express.js (`axios` or native `fetch`).
- **Database & Storage:** SQLite or PostgreSQL for storing user search history and saved repository summaries.
- **External APIs:** GitHub REST API (for fetching repository structure, README contents, and commit logs), OpenAI / Anthropic API (for text summarization).

4. Real-World Applications & Industry Relevance

- **Third-Party API Integration:** Demonstrates real-world skills in working with external REST APIs, handling rate limits, parsing JSON payloads, and managing authentication scopes.
- **AI-Driven Documentation Pipelines:** Mirrors modern developer productivity tools (such as automated changelog generators and repository briefing bots) utilized across enterprise engineering teams.
- **Full-Stack Data Persistence:** Exercises core database design patterns by saving user session activity and query history for future reference.

5. Architecture of the Project



- **Data Flow:** The user inputs a GitHub URL into the React frontend. The Express backend handles the request by calling the GitHub REST API to pull down the README markdown and commit logs. The backend then formats this text and passes it to the AI model to generate structured summaries before returning the payload to the frontend and saving the history record to the database.

6. One-Month Development Plan & Schedule

Week 1: Project Setup & Database Architecture

- **Milestones:**
    - Initialize the React frontend with Vite and Tailwind CSS.
    - Set up the Node.js and Express backend server, connecting it to SQLite or PostgreSQL.
    - Design database models for Users and Saved Repository Summaries (storing repo URLs, summaries, and timestamps).

- **Deliverables:** Working server connection, database schema setup, and base project layout.
-

Week 2: GitHub API Integration & URL Parser

- **Milestones:**
-
    - Build a robust regex helper on the backend to parse user input (extracting owner and repository names from github.com/owner/repo).
    - Implement API routes to fetch repository metadata, raw README content, and recent commit messages using the GitHub REST API.
    - Handle edge cases like private repositories, invalid URLs, and API rate limits.
- **Deliverables:** Functional backend service that successfully pulls data from public GitHub repositories.
-

Week 3: AI Summary Generator & Commit Digest

- **Milestones:**
-
    - Integrate the OpenAI or Anthropic SDK into the backend server.
    - Design system prompts to parse raw README files into structured highlights (Features, Prerequisites, Overview) and convert cryptic commit logs into clean release notes.
    - Build the frontend response view to nicely render markdown output.
- **Deliverables:** Working AI pipeline that processes repository data and returns structured summaries to the client.
-

Week 4: History Logs, UI Dashboard & Polish

- **Milestones:**
-
    - Build the user profile dashboard to display previously searched and saved repositories from the database.
    - Polish UI layouts, add loading spinners for async API calls, and handle error notification states.

    - Deploy the application (e.g., Vercel/Render).
    -
- **Deliverables:** Fully functioning, deployed MVP of GitSummarizer ready for portfolio presentation.
-

7. Helpful Resources & Documentation Links

Frontend & Styling

- React Official Documentation
-
- Tailwind CSS Documentation
-

Backend & External APIs

- Express.js Getting Started Guide
-
- GitHub REST API Documentation
-

Database & Storage

- SQLite Documentation
-
- PostgreSQL Documentation
-

AI Integration

- OpenAI Node.js API Reference
-

8. Conclusion

GitSummarizer bridges the gap between massive open-source codebases and quick developer understanding. By combining third-party REST API integration with AI text transformation, it provides a practical tool for modern developers while allowing builders to showcase full-stack proficiency in API orchestration, database management, and prompt engineering architecture.